

Building AI Application in Agriculture that knows your data: the RAG pipeline

Dr Sreenivasulu M

Data Science Manager

Publicis Sapient

Contents

- **The Context Challenge:** Understanding why AI needs instructions versus context.
- **RAG vs. Long Context:** The debate on whether large context windows replace the need for retrieval.
- **RAG Architecture & Algorithms:**
 - The Retrieve-Then-Generate Loop.
 - Sparse (Term-Based) vs. Dense (Embedding-Based) Retrieval.
 - Hybrid Search and Reciprocal Rank Fusion.
- **Optimization Strategies:**
 - Chunking Strategies and the Overlap Rule.
 - Query Rewriting for Disambiguation.
 - Contextual Retrieval (The Anthropic Approach).
- **Applications in Agriculture:**
 - **Kisan Call Center:** Managing government scheme documents.
 - **Soil Health Cards:** Handling tabular data with contextual retrieval.
 - **Crop Diagnosis:** Using Hybrid Search for symptoms and chemicals.
 - **Market Intelligence:** Agentic patterns for real-time data.

The Context Challenge

- The Core Problem:** AI models need two things to solve a task: instructions (how to do it) and information (context).
- Hallucination:** Just like humans, models hallucinate when they lack context.
- Context Engineering:** Constructing context for foundation models is the modern equivalent of "feature engineering" in classical ML.
- Two Dominating Patterns:**
 - 1.**RAG (Retrieval-Augmented Generation):** Retrieving relevant data from external sources.
 - 2.**Agents:** Using tools (search, APIs) to actively gather information and interact with the world.

Will Long Context Kill RAG?

- The Debate:** With 1M+ token windows, do we still need retrieval?

- Argument for RAG:**

- 1.**Data Velocity:** Data grows faster than context windows expand (Parkinson's Law of Data).

- 2.**Context Efficiency:** Models often focus on the wrong parts of massive contexts ("Lost in the Middle" phenomenon).

- 3.**Cost & Latency:** Every extra token costs money and adds latency. RAG reduces input to only the most relevant tokens.

- Anthropic's Rule of Thumb:** If knowledge is < 200k tokens (~500 pages), just use the prompt. For anything larger, use RAG. Its for claude models

- Note:** Model provider incorporates a retrieval like mechanism to help a model pick out the most salients parts of a context to use.



RAG Architecture Overview

- **History:** Concept introduced in 2017 (Chen et al.)- Reading Wikipedia to Answer open-domain questions.

- Term "RAG" coined in 2020 (Lewis et al.)- Knowledge intensive tasks. (they trained both generator and retrieval combinely)

- Enhancing generation by retrieving relevant information from external memory (databases, chat history, internet).

- **The Loop (Retrieve-Then-Generate):**

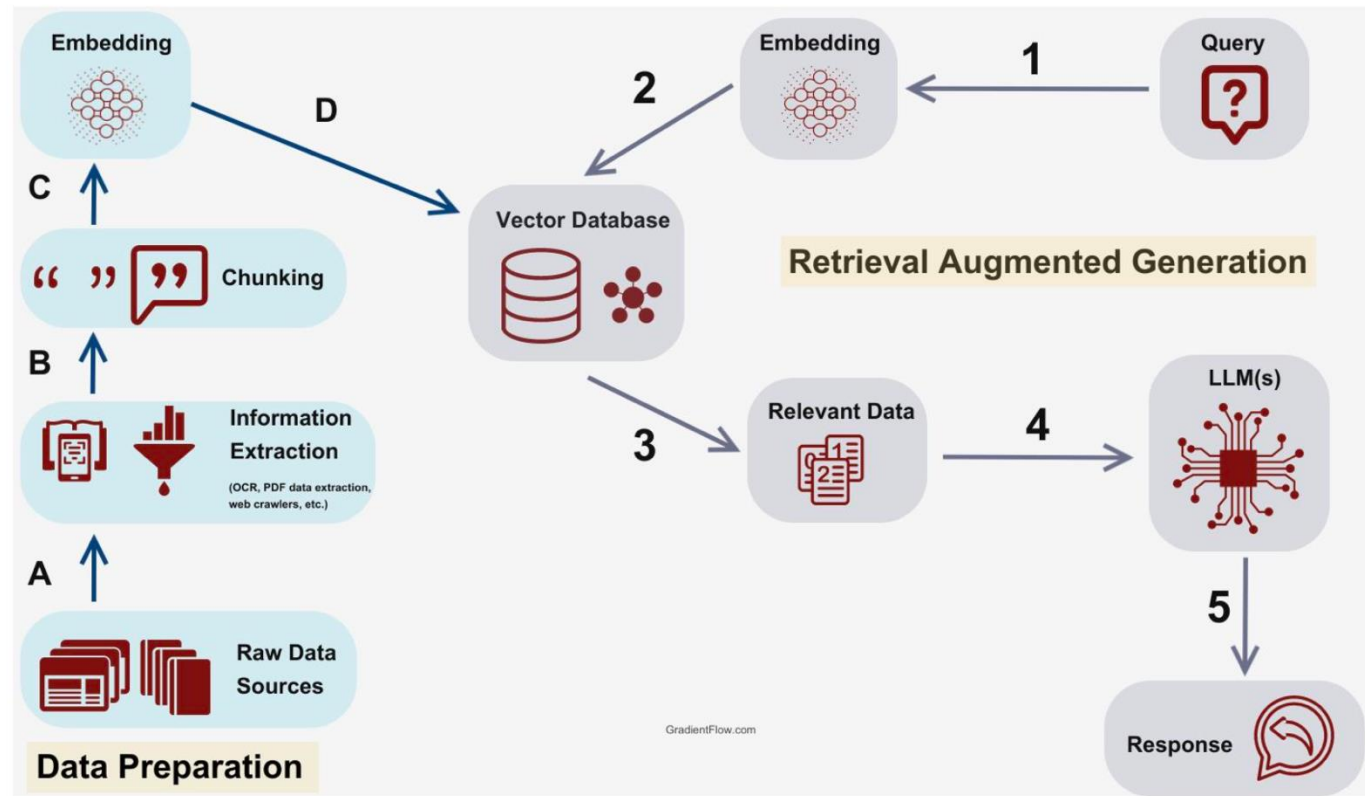
- 1. **Retriever:** Indexes and queries the external memory.

- 2. **Generator:** Uses the retrieved chunks + user prompt to generate the answer.

RAG (cont..)

- **Example query**
- Can Acme's fancy-printer-A300 print 100pps?
- Model responds better because of fancy-printer-A300

Architecture



Retrieval Algorithms

- Information is a century old idea.
- *1920's Emanuel Goldberg's patents for a statistical machine to search documents stored on films.*
- Backbone of search engines, recommender system, log analytics, etc.
- Two retrieval mechanisms-
 - Term-based retrieval
 - Embedding based retrieval.

Retrieval Algorithms I: Term-Based (Sparse)

- Mechanism:** Matches keywords using sparse vectors (mostly zeros).

- Key Metrics:**

- TF (Term Frequency):** How often a term appears in a document.

- IDF (Inverse Document Frequency):** Penalizes common words (like "the", "for").

- Algorithms:**

- BM25 (Best Matching 25)(1980):** The industry standard. Normalizes TF-IDF based on document length.

- Elasticsearch (2010):** Uses inverted indices for fast lookup.

- Pros/Cons:** Fast and cheap, but misses semantic meaning (e.g., "Apple" fruit vs. "Apple" company).

- Note:** Aravind Sreenivas, the ceo of perplexity tweeted “making a genuine improvement over BM25 is hard”

Retrieval Algorithms II: Embedding- Based (Dense)

- Mechanism:** Uses dense vectors to capture semantic meaning.

- Workflow:**

- 1.**Embedding Model:** Converts query and documents into vectors.

- 2.**Vector Database:** Stores embeddings (e.g., FAISS, Annoy, HNSW, Milvus).

- 3.**ANN Search:** Finds Approximate Nearest Neighbors.

- Pros/Cons:** Captures intent/synonyms, but computationally heavy and expensive to index.

Production Strategy: Hybrid Search & Reranking

- Hybrid Search:** Combining the precision of keyword matching (BM25) with the semantic understanding of embeddings.

- Reciprocal Rank Fusion (RRF):** An algorithm to combine ranked lists from different retrievers.

- Reranking:**

- Step 1:** Fetch 50+ candidates using a cheap retriever (e.g., BM25).

- Step 2:** Use a high-precision "reranker" model to sort them and select the top 5 for the LLM.

- Why?** Optimizes for both recall (don't miss data) and precision (don't distract the LLM).

Optimization I: Chunking Strategies

- **The Problem:** Documents vary in length (10 tokens vs 1M tokens).
- **Strategies:**
 - **Fixed Size:** e.g., 512 tokens with overlap.
 - **Recursive:** Split by section -> paragraph -> sentence.
 - **Semantic/Structural:** Split by markdown headers or code functions.
- **The Overlap Rule:** Always overlap chunks (e.g., 20 tokens) to prevent cutting context in the middle of a sentence (e.g., "I left my wife... a note").
- **Trade-off:** Smaller chunks = more diversity but less context. Larger chunks = better context but fewer fit in the window.

Optimization II: Query Rewriting

- **The Ambiguity Problem:**

- *User:* "When did John buy a hat?"

- *User:* "How about Emily?" -> **Context missing!**

- **Solution:** Use an LLM to rewrite the last query into a standalone question before retrieval.

- *Rewritten:* "When did Emily buy a hat?"

- **Impact:** Essential for conversational RAG to function correctly

Optimization III: Contextual Retrieval

- Technique:** Augmenting chunks with metadata or summaries.

- Anthropic's Approach:**

1. Use an LLM to generate a 50-100 token summary of the *entire document*.

2. Prepend this summary to *every chunk* of that document.

- Benefit:** Even small chunks carry the global context of the parent document, improving retrieval accuracy significantly

Evaluation Metrics

- **Component Evaluation:**
- **Context Precision:** % of retrieved documents that are relevant.
- **Context Recall:** % of *all* relevant documents that were retrieved.
- **Ranking Metrics:** NDCG (Normalized Discounted Cumulative Gain) and MRR (Mean Reciprocal Rank).
- **End-to-End Evaluation:** Judging the final generated answer quality.

Application of RAG in Agriculture

- Building Intelligent Text Interfaces for Farmers and Agronomists

The "Context" Problem in Agronomy

- **The Challenge:** To answer a farmer's question, a model needs specific instructions and **context**.
- **Missing Information:** Without access to specific local data (soil reports, local weather), AI models hallucinate answers.
- **The Solution:**
 - **RAG (Retrieval-Augmented Generation):** Retrieves relevant text (manuals, schemes) from an external database.
 - **Agents:** Uses tools to search the web for real-time news and prices

The "Kisan Call Center" RAG Bot

- **Use Case:** Answering queries about government schemes and subsidies.
- **Problem:** Government documents are vast. Long context windows are inefficient and models often focus on the wrong part of the text.
- **Architecture:**
 - **Retriever:** Fetches specific paragraphs from thousands of PDF scheme documents.
 - **Chunking Strategy:** Uses **recursive splitting** (Section -> Paragraph) to keep related text together.
 - **Result:** Precise answers about eligibility criteria without hallucinating non-existent benefits.

Handling Tabular Soil Health Cards

- Challenge:** Soil reports are often tables (Text + Numbers). Standard chunking breaks tables, losing context (e.g., a row of numbers without headers).
- Solution: Contextual Retrieval (The Anthropic Method).**
- Technique:** Use an LLM to generate a summary of the *whole* soil report (e.g., "Soil test results for Farmer X, Survey No. 123, 2024").
- Augmentation:** Prepend this summary to *every* chunk/row of the table.
- Outcome:** The retriever can successfully find "Nitrogen levels for Survey 123" even if the data was split into small chunks.

Crop Symptom Diagnosis (Hybrid Search)

- **Use Case:** Farmer describes symptoms: "Leaves are turning yellow with brown spots."
- **Engineering Challenge:**
- **Semantic Search:** Needed for symptoms (e.g., matching "yellowing" to "chlorosis").
- **Keyword Search:** Needed for specific chemical names (e.g., "Imidacloprid") or exact seed varieties.
- **Solution: Hybrid Search.**
- Combine **Embedding-based retrieval** (for symptoms) and **Term-based retrieval/BM25** (for chemical names) to find the correct protection manual.

The Market Intelligence Agent

- **Use Case:** "Is now a good time to sell Maize?"
- **Pattern:** Agentic Pattern.
- **Mechanism:**
 - The model uses **Tools** (APIs) to fetch external data (live Mandi prices, weather forecasts).
 - **Time-Based Reranking:** Retrieves news articles and reranks them to prioritize the most recent market updates.
- **Impact:** Provides advice based on *live* conditions, not training data from 2 years ago.

Optimization: Query Rewriting for Chatbots

- Turn 1 (User):** "What is the recommended dosage of **Coragen** for stem borers?"
- System Response:* "The recommended dosage is 60ml per acre."
- Turn 2 (User):** "Can I mix **it** with a fungicide?"
- The Problem:** The user uses the pronoun "**it**". If the RAG system searches the database for "Can I mix **it** with a fungicide?", it will likely retrieve generic mixing guides or irrelevant results because "it" is undefined.
- The Rewrite:** The system uses an LLM to rewrite the query before sending it to the retriever:
- Rewritten Query:** "Can I mix **Coragen** with a fungicide?"

Summary

- **NLP in Agriculture is an Engineering Problem:** It requires careful context construction.

- **Key Takeaways:**

- Use **Contextual Retrieval** for Soil/Lab reports.

- Use **Hybrid Search** to handle both symptoms (semantic) and chemical names (keywords).

- Use **Agents** for real-time decision-making (markets/weather).

Thank you

